

Disentangling Parser and Data Effects in Tool-Using LLM Fine-Tuning: A Case Study on Qwen2.5-7B

Alyssa L

Department of Information Science and Engineering
Siddaganga Institute of Technology, Tumakuru, Karnataka, India
alyssal282006@gmail.com

Abstract

End-to-end benchmark scores for tool-using language-model agents mix model capability, training data, and serving-layer and client infrastructure. We report a small case study on a fixed 23-task, machine-checkable multi-tool benchmark. The untuned Qwen2.5-7B-Instruct baseline scored 22/23 (95.7%). QLoRA fine-tuning on 35 hand-curated trajectories (condition **V1**) regressed performance to 18/23 (78.3%). We then re-evaluated the *unchanged V1 adapter* behind a corrected two-pass tool-call parser (condition **E4**), and E4 also scored 18/23, with four task outcomes flipping: two recovered (`multi_three_tools`, `adversarial_false_premise`) and two newly lost (`multi_search_weather`, `adversarial_chained_search_calc`). A post-hoc audit of the artifacts showed that E4 was scored with a different, notebook-local evaluation client. Its tool schemas, tool-call and tool-result serialization, and tool implementations all differed from the other conditions, and it ran two weeks later against live data. The V1→E4 contrast therefore changes the parser *and* the client, and it does not identify a parser-only effect. The V1 traces also show that the two “recovered” tasks did not fail because of dropped tool calls. A further intervention (condition **V2**) added 11 trajectories written after inspecting V1’s failures on this benchmark, and retrained with different memory-related optimizer settings. V2 reached 23/23. We report this as an observation about a failure-informed intervention, not as a causal or generalizing result. The main lesson is methodological. Equal aggregate scores concealed per-task changes, and the infrastructure held “constant” between conditions has to be verified from artifacts rather than assumed. We release the trajectories, serving code, benchmark definitions, result files, and the notebook used for E4.

Keywords: tool use, function calling, QLoRA, fine-tuning regression, ablation study, parser reliability, agent evaluation, benchmark adaptation.

1 Introduction

Large language models (LLMs) are increasingly deployed as agents that select tools, emit structured calls, receive tool outputs, and continue reasoning. The end-to-end score re-

ported by any such system is a composite of many stages: prompt construction, model decoding, serialization of the tool call, extraction by a serving-layer parser, execution by the tool, and finally interpretation of the returned observation by the same model. When a benchmark score changes after an intervention—fine-tuning, prompt editing, parser change, or tool change—the aggregate number alone does not reveal which stage was responsible.

This paper documents a concrete instance of that conundrum in a small, controlled setting. We fine-tuned Qwen2.5-7B-Instruct with QLoRA (Dettmers et al., 2023) on 35 hand-curated tool-use trajectories and observed a regression from the untuned baseline of 22/23 to 18/23 on a fixed 23-task benchmark. We considered two candidate causes: (i) a single-pass regex parser that could silently drop malformed but semantically correct tool calls, observed in a separate diagnostic run, and (ii) insufficient training coverage of chained tool use, multi-tool synthesis, and error handling.

We then replaced the parser (holding the fine-tuned adapter and training data fixed) and re-evaluated. The resulting condition, **E4**, scores 18/23: identical to V1 in aggregate but composed of a different set of per-task outcomes. Two tasks were recovered and two broke. As Section 5.5 documents, E4 was also scored with a different evaluation client, so these flips cannot be attributed to the parser alone. A subsequent condition, **V2**, retained the corrected parser and added 11 targeted training trajectories, achieving 23/23.

Our contributions are:

1. An adapter-fixed ablation (E4) in which the serving-layer parser and, as run, the evaluation client changed. The aggregate score was unchanged while four task outcomes flipped.
2. A concrete case in which aggregate accuracy was invariant while failure composition changed, together with an artifact-level audit showing that an intended single-factor ablation silently varied a second factor.
3. An honest, reproducible record of a fine-tuning regression, its trace-level diagnosis, and a subsequent intervention, with the iterative nature of benchmark development disclosed.
4. An open artifact package containing the training trajectories, serving code for both parsers, the 23-task

benchmark, the scoring harness, and all result files.

We deliberately avoid claiming that fine-tuning helped, that the parser mattered in general, or that 23/23 implies generalization. The contribution is methodological and empirical within a small, clearly bounded setting.

2 Research Questions and Problem Formulation

2.1 Problem formulation

Consider a tool-using agent that, given a user task x , produces a sequence of turns. At each turn t , the model emits an output string y_t which may contain one or more tool calls. A parser P maps y_t to a set of structured calls $C_t = P(y_t)$. Each call is dispatched to a tool; the tool returns an observation o_t ; the observation is appended to the model’s context; and the loop continues until the model emits a final answer or a step budget S is exhausted.

A benchmark task i defines a required tool set R_i , a set of content or numeric predicates on the final answer, and possibly a constraint that a tool error must (or must not) occur. Let $\text{score}_i \in \{0, 1\}$ denote the harness’s verdict on task i under parser P and adapter A . The aggregate accuracy is

$$\text{Acc}(A, P) = \frac{1}{N} \sum_{i=1}^N \text{score}_i(A, P). \quad (1)$$

We are interested in the contrast between two configurations that share an adapter but differ in parser:

$$\Delta_{\text{parser}}(A) = \text{Acc}(A, P_{V2}) - \text{Acc}(A, P_{V1}). \quad (2)$$

The design *intended* to estimate Δ_{parser} for the adapter A_{V1} trained on 35 trajectories; this is the E4 comparison. As run, E4 also changed the evaluation client E , so the observed contrast is $\text{Acc}(A_{V1}, P_{V2}, E_{\text{nb}}) - \text{Acc}(A_{V1}, P_{V1}, E_{\text{mcp}})$ rather than Δ_{parser} (Section 5.5). A separate contrast,

$$\Delta_{\text{data}} = \text{Acc}(A_{V2}, P_{V2}) - \text{Acc}(A_{V1}, P_{V2}), \quad (3)$$

measures the combined effect of changing the adapter and adding 11 targeted trajectories, holding the parser fixed. This is the E4→V2 comparison. The second contrast is *not* a randomized causal estimate: the added trajectories were authored using knowledge of V1’s failures.

2.2 Research questions

RQ1. On a fixed benchmark, what is the net aggregate effect of replacing the V1 parser with the V2 two-pass parser while holding the V1 adapter and training data constant? (As run, E4 answers a weaker version of this question; see Section 5.5.)

RQ2. Does that replacement change the composition of per-task failures even when the aggregate does not change?

RQ3. What is the observed change when targeted trajectories are added after the parser replacement, and what are the validity conditions under which that change can be interpreted?

RQ4. Which of these results can be generalized beyond the evaluated benchmark, and which cannot?

3 Related Work

We organize prior work into five threads that bear directly on this study.

3.1 Tool-using and function-calling language models

ReAct (Yao et al., 2022) established the interleaved reason–act–observe loop that motivates most modern agent designs. Toolformer (Schick et al., 2023) showed that a language model can self-supervise the placement of API calls via a generate-and-filter procedure. Gorilla (Patil et al., 2023) fine-tuned LLaMA-7B for accurate invocation across 1,600+ APIs using retrieval. ToolLLM (Qin et al., 2023) scaled demonstration data to 16,000+ APIs via depth-first search over tool compositions. API-Bank (Li et al., 2023) introduced a benchmark of 11,428 samples across 53 APIs, explicitly separating API selection from argument generation.

These works established both the capability and the evaluation landscape but largely assume a correct parser between model output and tool execution. Our study treats the parser as part of the evaluated system.

3.2 Agent evaluation

AgentBench (Liu et al., 2023) evaluates LLM agents across 11 interactive environments. WebArena (Zhou et al., 2023) evaluates agents on realistic web tasks. Common to these benchmarks is an end-to-end aggregate score that does not, in itself, decompose model-level errors from serving-layer or protocol-level errors.

Our benchmark is far smaller (23 tasks, single tool surface) and its purpose is orthogonal: to permit exhaustive trace-level attribution of every failure, at the cost of statistical strength.

3.3 Agent fine-tuning

FireAct (Chen et al., 2023) fine-tuned language agents on ReAct-style trajectories and reported improvements on HotpotQA and ALFWorld. AgentTuning (Zeng et al., 2023) curated 37k multi-turn trajectories and showed gains on held-out agent tasks. These works demonstrate that fine-tuning can improve agent behavior at scale. They do not typically report controlled ablations that hold the adapter fixed while varying serving infrastructure.

Our study operates at a much smaller data scale (35 and 46 trajectories) and focuses on failure dynamics rather than scaling laws.

3.4 Parameter-efficient fine-tuning

LoRA (Hu et al., 2021) and QLoRA (Dettmers et al., 2023) make 7B-scale fine-tuning feasible on a single consumer or free-tier GPU. We use a shared QLoRA configuration across conditions, so that the differences we observe are confined to the adapter, the training data, and the parser—with one exception we state explicitly in Section 5, namely

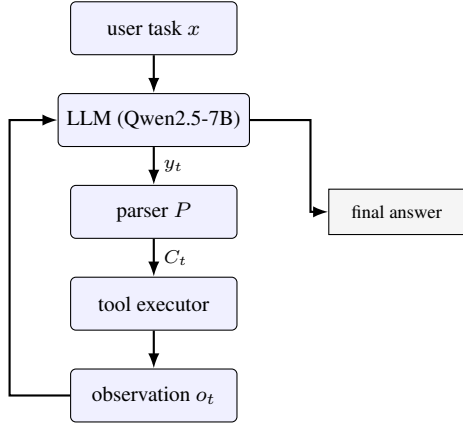


Figure 1: The AGENTRUN pipeline. The parser sits between model output and tool execution; failures at the parser stage are indistinguishable at the aggregate score from failures at the model or tool stages.

three memory-related optimizer settings that the V2 training script adds.

3.5 Evaluation reliability, structured output, and benchmark adaptation

Structured generation. Willard and Louf (Willard and Louf, 2023) proposed finite-state-machine-guided generation for structured output. Related work treats the reliability of serializing structured model output as a first-class engineering problem. Our parser is a lightweight, hand-written alternative to constrained decoding and has known failure modes we characterize.

Contamination and adaptive evaluation. Sainz et al. (2023) and Deng et al. (2023) documented the ways in which benchmark results can be inflated by data contamination or by iterative benchmark development. Our study is not contaminated in the classical sense—no benchmark prompt appears in the training data—but the V2 augmentation was authored with knowledge of which benchmark tasks V1 failed. This is a mild form of evaluation adaptation, and we disclose it explicitly.

3.6 Positioning

To our knowledge, no prior study on this scale has reported a controlled ablation in which the serving-layer tool-call parser is varied while the adapter and training data are held fixed, with per-task outcomes reported alongside the aggregate.

4 The AGENTRUN System

4.1 Pipeline

Figure 1 shows the AGENTRUN pipeline. The model emits output y_t , the parser extracts tool calls C_t , the executor runs them, the observations o_t are appended to the context, and the loop repeats until a final answer or the step budget is exhausted.

4.2 Model and adapters

The base model is Qwen/Qwen2.5-7B-Instruct (Qwen Team, 2024) (7.6B parameters), served in 4-bit NF4 quantization with fp16 compute. LoRA adapters are applied with rank $r = 16$, $\alpha = 16$, dropout 0.05, target modules $\{q_proj, k_proj, v_proj, o_proj\}$. Two adapters exist:

- **V1:** trained on 35 hand-curated trajectories.
- **V2:** trained on 46 trajectories, i.e., the V1 set plus 11 targeted trajectories.

Both adapters share the same LoRA, data, and optimization hyperparameters; the three memory-related settings that differ are listed in Table 2.

4.3 Tool surface

The tool surface is fixed across all conditions and consists of exactly four tools: `calculator`, `wordcount`, `web_search` (Tavily API), and `get_weather` (Open-Meteo). Tool calls are transported via a Model Context Protocol server over stdio. The tool surface and tool schemas presented to the model are byte-identical across all conditions; only the model and the parser vary.

4.4 Parser variants

Two parsers are used. Both consume the raw string output of the model and return a dictionary `{"tool_calls": [...], "text": ...}`.

V1 parser (single-pass). A single regular expression matches properly closed `<tool_call>...</tool_call>` blocks:

```

TOOL_CALL_PATTERN = re.compile(
    r"<tool_call>\s*({\s*.*?})\s*</tool_call>",
    re.DOTALL)

```

If the model emits a malformed block—e.g., an unclosed opening tag followed by a second opening tag—the regex’s non-greedy match spans across the malformed boundary, fails to parse the captured substring as JSON, and returns no tool calls. The server then treats the entire raw output as a final answer.

V2 parser (two-pass). A first pass attempts strict matching, identical to V1. If the strict pass finds nothing, a second pass extracts the first JSON object occurring after any `<tool_call>` opening tag. The second pass has a known limitation: it can only recover the *first* tool call when the model emits multiple malformed blocks in a single turn, and it may truncate arguments that contain nested JSON.

4.5 Experimental conditions

Four conditions are compared (Table 1).

The E4 condition was designed to hold the adapter and training data equal to V1 and change only the parser to the V2 version. As run, it also used a different evaluation client (Section 5.5).

Table 1: Experiment matrix. E4 is the controlled parser ablation.

ID	Adapter	Training data	Parser	Score
Base	none	none	V1	22/23
V1	V1	35 trajectories	V1	18/23
E4	V1	35 trajectories	V2	18/23
V2	V2	46 trajectories	V2	23/23

Table 2: Training configuration. All rows are shared between V1 and V2 except the three memory-related settings, which are added in the V2 script.

Hyperparameter	Value
Base model	Qwen/Qwen2.5-7B-Instruct
Quantization	4-bit NF4, fp16 compute
LoRA rank r	16
LoRA α	16
LoRA dropout	0.05
Target modules	<code>q, k, v, o_proj</code>
Optimizer	V1: default; V2: PagedAdamW8bit
Gradient ckpt.	V1: off; V2: on
KV cache	V1: on; V2: off
Learning rate	2×10^{-4}
Per-device batch	2
Gradient accum.	4
Effective batch	8
Epochs	3
Max seq. length	1024
Hardware	Colab T4 (15 GB)

5 Experimental Setup

5.1 Training configuration

Table 2 lists the training configuration of both runs. Every LoRA and data hyperparameter is shared. Three memory-related settings are not: the V2 script additionally enables gradient checkpointing, disables the KV cache, and passes a paged 8-bit AdamW optimizer explicitly, because the 46-trajectory run had to fit in the 15 GB budget of a free-tier T4. The V1 script does not set these and therefore uses the `Trainer` default optimizer. This difference does not enter the V1→E4 contrast, because E4 reuses the V1 adapter, but it does mean that the E4→V2 contrast changes the adapter, the training data, and these optimizer settings simultaneously (Section 9).

5.2 Training trajectories

V1 (35 trajectories). Generated by running the untuned base agent over a set of training prompts and retaining only trajectories that completed within the step budget without tool errors. Training prompts are disjoint from benchmark prompts.

V2 (46 trajectories). The V1 set plus 11 targeted additions (Table 3).

The choice of *which* patterns to add was informed by inspecting which benchmark categories V1 failed. This is disclosed and analyzed in Sections 10 and 11.

Table 3: V2 targeted augmentation (11 additional trajectories).

Targeted gap	Trajectories added
<code>word.count</code> → <code>calculator chains</code>	3
<code>get.weather</code> → <code>calculator chains</code>	3
Three-tool chain with final synthesis	2
Calculator error handling (division by zero)	2
False-premise identification and correction	1
Total	11

Table 4: Benchmark task categories (23 tasks).

Category	#	What it probes
<code>math</code>	3	Arithmetic, precedence
<code>text</code>	1	Word counting
<code>weather</code>	2	Live weather lookup
<code>search</code>	2	Live web search
<code>multitool</code>	4	Independent tools, one turn
<code>chained_reasoning</code>	1	Tool B depends on A
<code>time_sensitive</code>	1	Live info, not parametric
<code>no_tool_expected</code>	2	Withholding tool use
<code>adversarial_math</code>	3	Negatives, decimals, %
<code>adversarial_reasoning</code>	2	False premise; chain
<code>adversarial_error_handling</code>	1	Tool must error (div.)
<code>adversarial_ambiguity</code>	1	Underspecified entity
Total	23	

5.3 Benchmark

The benchmark consists of 23 machine-checkable tasks (Table 4).

Each task’s success predicate is checked against the full execution trace: (i) required tools present in the trace’s tool-name set, (ii) no unexpected tool error unless the task explicitly requires one, (iii) termination within `max_steps = 3`, and (iv) any content or numeric predicate satisfied by the final answer.

5.4 Evaluation protocol

All four conditions were evaluated with the same benchmark, the same scoring function, `max_steps = 3`, and greedy decoding (`do_sample=False`). The Tavily API key was active for all reported runs, and no tool is mocked. Base, V1, and V2 were run on 2026-09-01 with the local client `ft_agent.py`, which executes tools through an MCP server. E4 was not.

5.5 How E4 was actually evaluated

The reported E4 score comes from an evaluation loop written inside the Colab notebook (`notebooks/colab_e4_session.ipynb`, cell 16), run on 2026-09-16. That loop imports the same tasks and `score_task` and uses `max_steps = 3`, but it differs from `ft_agent.py` in four ways:

- its tool schemas carry minimal descriptions;
- tool calls are returned to the model as OpenAI-style `tool_calls` and tool results as `{"result":`

...} JSON, instead of `<tool_call>` text and provider-formatted strings;

3. tools are notebook-local functions, and only exceptions count as tool errors;
4. there is no client-side re-parse of the model text.

Every multi-step task therefore saw a different context from step 2 onward. Live search and weather results also differed between the two dates. The notebook additionally contains an E4 run made without a working search key, which scored 13/23. The reported run is the one with the key working. A client-matched E4 (evaluation/run_eval_e4.py) was not run.

6 Failure Analysis

V1 and V2 recorded full traces; E4 recorded pass/fail only. We therefore describe V1 behavior from its traces and make no mechanism claim for E4.

6.1 Tasks that flipped between V1 and E4

multi_three_tools (V1 fail → E4 pass). In V1 the model called all three required tools (`get_weather`, `calculator`, `web_search`) but used all three steps and hit the step budget. No call was dropped by the parser.

adversarial_false_premise (V1 fail → E4 pass). In V1 the model called `web_search` and then answered without acknowledging that the country is fictional, which failed the content predicate. No call was dropped by the parser.

multi_search_weather, adversarial_chained_search_calc (V1 pass → E4 fail). In V1 both tasks called all required tools and passed.

The four flips are what was observed when the parser and the evaluation client changed together. Neither the V1 traces nor the E4 artifacts identify the parser as their cause.

6.2 Failures shared by V1 and E4

multi_wordcount_calc. In V1 the model calls `wordcount` but performs the multiplication in prose rather than with the calculator, which fails the required-tool check.

chain_weather_then_calc. In V1 the model calls `get_weather` but computes the Celsius-to-Fahrenheit conversion in text.

adversarial_division_by_zero. The task requires a calculator call that raises an error. In Base and V1 the model answers in prose without calling the calculator.

Table 5: Aggregate accuracy across conditions.

Condition	Adapter	Parser	Passed	Accuracy
Base	none	V1	22/23	95.7%
V1	V1 (35)	V1	18/23	78.3%
E4	V1 (35)	V2	18/23	78.3%
V2	V2 (46)	V2	23/23	100.0%

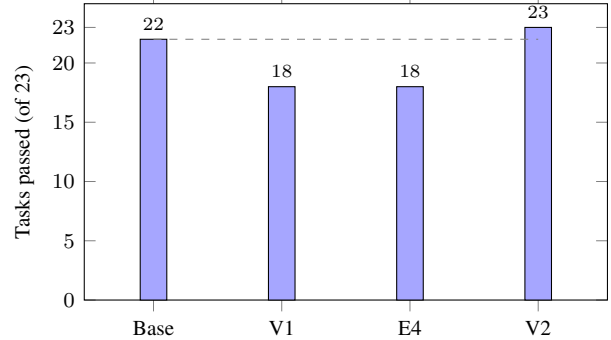


Figure 2: Aggregate pass counts. Dashed line marks Base. E4 matches V1 in aggregate; V2 exceeds both.

7 Results

Two contrasts follow; neither isolates a single factor:

- **V1→E4 (parser and client changed):** 18/23 — 18/23 = 0 net tasks, with 2 recovered and 2 regressed.
- **E4→V2 (adapter retrained; client also differs):** 23/23 — 18/23 = +5 tasks.

7.1 Per-task decomposition

Table 6 reports every benchmark task under all four conditions.

8 Adapter-Fixed Ablation (E4)

*On the 23-task benchmark, with the V1 adapter and training data fixed, replacing the V1 single-pass parser with the V2 two-pass parser; together with the change of evaluation client described in Section 5.5, produced **zero net change in aggregate accuracy** (18/23 → 18/23).*

Two qualifications apply:

1. Specific tasks changed. Two tasks that V1 failed were recovered, and two tasks that V1 passed regressed.
2. The cause of each flip is not identified. E4 recorded no traces, and the parser, the client, and the live tool data all changed between the two runs.

9 Data-Augmentation Analysis

The E4→V2 transition retains the corrected parser and changes the adapter from V1 (35 trajectories) to V2 (46 trajectories). It recovers all five E4 failures. We state this as

Table 6: Per-task outcomes across all conditions. $\checkmark$ = pass, $\times$ = fail. Columns Δ_1 and Δ_2 record $V1 \rightarrow E4$ and $E4 \rightarrow V2$ changes.

Task ID	Category	Base	V1	E4	V2	Δ_1	Δ_2
calc.basic.1	math	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	-	-
calc.basic.2	math	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	-	-
calc.order.of.ops	math	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	-	-
wordcount.basic	text	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	-	-
weather.basic.1	weather	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	-	-
weather.basic.2	weather	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	-	-
search.basic.1	search	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	-	-
search.basic.2	search	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	-	-
multi.weather.calc	multi	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	-	-
multi.wordcount.calc	multi	$\checkmark$	$\times$	$\times$	$\checkmark$	-	+1
multi.search.weather	multi	$\checkmark$	$\checkmark$	$\times$	$\checkmark$	-1	+1
multi.three.tools	multi	$\checkmark$	$\times$	$\checkmark$	$\checkmark$	+1	-
chain.weather.then.calc	chained	$\checkmark$	$\times$	$\times$	$\checkmark$	-	+1
time.sensitive.1	time	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	-	-
no.tool.needed.1	no-tool	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	-	-
no.tool.needed.2	no-tool	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	-	-
adversarial.negative.multiply	adv-math	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	-	-
adversarial.decimal.division	adv-math	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	-	-
adversarial.percentage	adv-math	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	-	-
adversarial.false.premise	adv-reas	$\checkmark$	$\times$	$\checkmark$	$\checkmark$	+1	-
adversarial.chained.search.calc	adv-reas	$\checkmark$	$\checkmark$	$\times$	$\checkmark$	-1	+1
adversarial.division.by.zero	adv-err	$\times$	$\times$	$\times$	$\checkmark$	-	+1
adversarial.ambiguous.city	adv-amb	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	-	-
Passed		22	18	18	23	0	+5

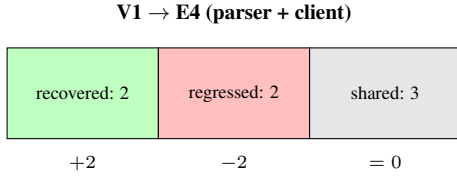


Figure 3: Per-task transitions from V1 to E4 (parser and evaluation client changed). Recoveries and regressions cancel exactly.

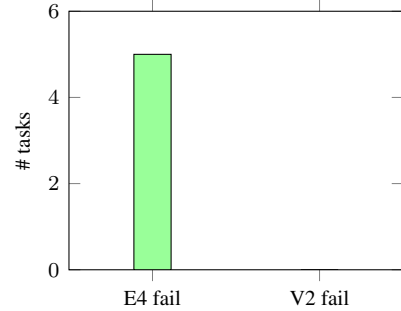


Figure 4: $E4 \rightarrow V2$ transition: all five E4 failures recovered; none regressed.

an observation, not as a randomized causal estimate. Four reasons:

1. The 11 additional trajectories were selected because V1 failed on those patterns. This is a form of evaluation adaptation.
2. The adapter change and the training-data change are inseparable here.
3. The V2 training script also enables gradient checkpointing, disables the KV cache, and uses a paged 8-bit optimizer that the V1 script does not (Table 2), so the two runs are not separated along the training axis alone.
4. No seeds, no repetitions, and no held-out benchmark are available in this study.
5. E4 and V2 were scored with different evaluation clients (Section 5.5).

The defensible statement is: on the evaluated benchmark, retraining on 46 trajectories (the V1

set plus 11 targeted additions) under the corrected parser, with the V2 run's memory-related optimizer settings, coincides with recovery of all five remaining failures. Whether the same intervention would generalize is not tested.

10 Discussion

10.1 Interpreting E4

E4 was designed as the cleanest contrast in the study, but as run it is not. On the 23-task benchmark, with the V1 adapter fixed, changing both the parser and the evaluation client produced zero net change in aggregate accuracy while four tasks flipped. Because the client changed the prompt context of every multi-step task, and the flipped tasks are

all multi-step, the client is at least as plausible a cause as the parser. A client-matched re-run is the necessary next experiment.

10.2 Interpreting the E4→V2 transition

The E4→V2 transition retains the corrected parser and changes the adapter. It recovers all five E4 failures. We state this as an observation, not as a randomized causal estimate, for the reasons given in Section 9.

10.3 Aggregate invariance and failure redistribution

The V1→E4 result illustrates a general point for agent evaluation: an aggregate score can be invariant while the underlying failure composition changes. A deployment decision that treats the two conditions as equivalent because the aggregate is equal would be incorrect if the specific tasks that fail matter.

10.4 Why fine-tuning can regress

V1’s drop from 22/23 to 18/23 is not explained by parser drops in the recorded V1 run. Its five failures are a missing calculator call (two tasks), step-budget exhaustion, missing false-premise wording, and not calling the calculator for division by zero. The study does not identify whether this is catastrophic forgetting, overfitting, or a coverage gap; it identifies the patterns that regressed and the training data that did not cover them.

11 Threats to Validity

Internal validity. The E4 comparison holds the adapter and training data fixed, but the evaluation client, the tool implementations, and the run date also changed (Section 5.5), so it is not a single-factor contrast. The E4→V2 comparison is not a clean causal contrast: the added data was authored using knowledge of V1 failures, and the V2 training script additionally changes the optimizer and gradient-checkpointing settings (Table 2). Environmental variance from live web search is uncontrolled.

External validity. Results are on a single model family (Qwen2.5-7B-Instruct) and a single 23-task benchmark with four tools. Generalization to other models, benchmarks, or tool surfaces is not tested.

Construct validity. The scoring harness combines multiple stages of behavior into a single binary outcome. Distinguishing “model failed to select the tool” from “model selected the tool but the parser dropped the call” requires trace inspection that the harness does not perform.

Statistical validity. One run per condition. No variance estimate. No significance test.

Evaluation validity. The 23-task benchmark was used iteratively during development. The V2 augmentation was designed after observing V1’s failures. A fresh held-out benchmark is required for a generalization claim; we do not make one.

Reproducibility validity. The local evaluation environment is recorded (Python 3.11.9, torch 2.13.0+cpu, transformers 5.15.1, peft 0.20.0, accelerate 1.14.0). The Colab training and serving package versions, CUDA version, and driver version were not recorded. Random seeds for training and evaluation were not set.

12 Limitations

- **N=1 per condition.** No multi-seed runs. Variance is unknown.
- **Single model family.** All results are Qwen2.5-7B-Instruct.
- **Small benchmark.** 23 tasks; six categories have N=1.
- **Iterative benchmark development.** V2’s training data was informed by V1’s failures on this exact benchmark.
- **No fresh held-out benchmark.** Not run.
- **Retraining is not isolated.** The V2 run differs from V1 in training data and in three memory-related optimizer settings; no run isolates the data change on its own.
- **Greedy decoding only.**
- **Real external-tool variability.**
- **E4 is not client-matched.** E4 used a notebook-local evaluation loop and tools, and it records pass/fail only, with no traces.
- **Uneven predicate strength.** Seven tasks check only that tools were called, and one no-tool task has no positive check.

The 23/23 V2 result means “23/23 on the evaluated benchmark.” It does not mean the model generalizes perfectly, that the augmentation caused the improvement in a randomized sense, or that the parser fix mattered (or did not matter) in general.

13 Reproducibility and Artifact Availability

All artifacts are available at <https://github.com/alyssa2ai/AGENTRUN>:

- `eval/tasks.py` — 23-task benchmark definitions.
- `eval/harness.py` — scoring logic.
- `training/training_data.jsonl` — V1 trajectories (35).
- `training/training_data_augmented.jsonl` — V2 trajectories (46).
- `serving/colab_server.py`, `serving/colab_server_e4.py`, `serving/colab_server_v2.py` — serving code.

- `results/eval_results_{base,v1,e4,v2}.json` — raw per-condition results.
- `results/experiment_summary.json` — full matrix and provenance.
- `results/verify_results.py` — recomputes every reported tally and transition from the committed result files.
- `provenance/experiment_provenance.json` — provenance record with per-field verification labels.

Missing metadata (flagged rather than guessed): exact Colab CUDA and driver versions; training and evaluation random seeds; exact wall-clock timestamp of the E4 run.

14 Conclusion

We presented a small case study of fine-tuning a tool-using LLM agent. On the evaluated 23-task benchmark, with the fine-tuned adapter fixed, replacing the V1 single-pass parser with a V2 two-pass parser, together with an unplanned change of evaluation client, produced zero net aggregate change while four per-task outcomes flipped. A subsequent failure-informed intervention, with 11 targeted trajectories and retraining, recovered all five remaining failures. Neither contrast isolates a single factor. Both are benchmark-specific observations that do not support causal or generalizing claims.

Three recommendations follow. First, when multiple changes are made between reported conditions, run a controlled ablation that isolates each change, and verify from the artifacts, not from the plan, that everything else was held fixed. Second, report per-task outcomes alongside aggregate scores; equal aggregates can hide opposite-sign per-task effects. Third, treat targeted training-data augmentation as a legitimate but not blindly causal intervention; if the augmentation was informed by observed failures on the evaluation set, disclose that.

Future work should include a client-matched E4 re-run, multi-seed evaluation, a fresh held-out benchmark with novel task instances, a random-augmentation control, and cross-model replication. None of these are prerequisites for the claim we make; they are prerequisites for a stronger claim we deliberately do not make.

References

- Baian Chen, Chang Shu, Ehsan Shareghi, Nigel Collier, Karthik Narasimhan, and Shunyu Yao. FireAct: Toward language agent fine-tuning, 2023.
- Chunyu Deng, Yanchen Zhao, Xiangru Tang, Mark Gestein, and Arman Cohan. Investigating data contamination in modern benchmarks for large language models, 2023.
- Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized llms, 2023.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models, 2021.
- Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. API-Bank: A comprehensive benchmark for tool-augmented llms, 2023.
- Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, et al. AgentBench: Evaluating llms as agents, 2023.
- Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large language model connected with massive apis, 2023.
- Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, et al. ToolLLM: Facilitating large language models to master 16000+ real-world apis, 2023.
- Qwen Team. Qwen2.5 technical report, 2024.
- Oscar Sainz, Jon Ander Campos, Iker García-Ferrero, Julen Etxaniz, Oier Lopez de Lacalle, and Eneko Agirre. NLP evaluation in trouble: On the need to measure LLM data contamination for each benchmark, 2023.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools, 2023.
- Brandon T. Willard and Rémi Louf. Efficient guided generation for large language models, 2023.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models, 2022.
- Aohan Zeng, Mingdao Liu, Rui Lu, Bowen Wang, Xiao Liu, Yuxiao Dong, and Jie Tang. AgentTuning: Enabling generalized agent capabilities for llms, 2023.
- Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, et al. WebArena: A realistic web environment for building autonomous agents, 2023.